

YOLO v10 원도 설치 및 실행

JD-edu

2025 All right Reserved

YOLOv10 이론 개요

YOLOv10은 YOLO(You Only Look Once) 계열의 최신 객체 탐지 모델로, 이전 버전들의 장점을 계승하고 효율성과 성능을 극대화한 것이 특징입니다.

YOLO란?

실시간 객체 탐지 분야에서 혁신을 가져온
단일 단계(One-Stage) 모델입니다.

핵심 특징

이미지 전체를 한 번의 순방향 신경망
실행만으로 탐지합니다.

차별점

전통적인 2단계 방식과 달리 영역 제안 없이
즉시 탐지를 수행합니다.

YOLO의 CNN 기반 작동 원리

YOLO 모델은 **CNN(Convolutional Neural Network)**을 핵심 백본으로 사용하여 이미지의 특징을 추출하고, 이를 통해 객체를 탐지하고 분류합니다.

01

이미지 분할

입력 이미지를 $S \times S$ 크기의 격자(Grid Cell)로 나눕니다. 각 격자 셀은 해당 영역 내에 객체의 중심이 위치할 경우 그 객체를 탐지할 책임을 갖습니다.

02

특징 추출

CNN 백본 네트워크가 입력 이미지 전체를 스캔하며 계층적인 특징을 추출합니다. 초기 레이어는 엣지와 색상을, 깊은 레이어는 높은 레벨의 의미론적 특징을 추출합니다.

특징 융합 및 예측 수행

Neck 네트워크

백본에서 추출된 여러 스케일의 특징 맵을 Neck 네트워크(FPN, PANet, BiFPN)가 통합하고 강화합니다.

- 다중 스케일 특징 융합으로 큰 객체와 작은 객체 모두 효과적으로
- 뿔뿔식과 하향식 경로를 통해 저수준 특징과 고수준 특징을 융합

Detection Head

각 격자 셀마다 세 가지 정보를 동시에 예측합니다:

바운딩 박스 좌표: (x, y, w, h) 형태로 표현

객체 신뢰도: $P(\text{Object}) \times \text{IOU}$ 계산

클래스 확률: 각 클래스에 속할 조건부 확률

최종 탐지 결과 생성

1

클래스별 신뢰도 계산

각 바운딩 박스에 대해 클래스별 최종 신뢰도 점수를 계산합니다.

2

임계값 필터링

신뢰도 점수가 임계값 이하인 박스들을 제거하여 잘못된 탐지를 걸러냅니다.

3

NMS 적용

중복된 바운딩 박스를 제거하여 하나의 객체에 하나의 박스만 남깁니다.

4

최종 출력

바운딩 박스 좌표, 클래스 레이블, 신뢰도 점수를 포함한 최종 결과를 출력합니다.

- ❏ 이러한 전체 과정이 단일 신경망 순방향 패스(Single Forward Pass) 내에서 이루어지기 때문에 YOLO는 매우 빠른 추론 속도를 자랑하며, 실시간 객체 탐지에 최적화되어 있습니다.

YOLO가 하는 일

YOLO 모델의 주요 임무는 실시간으로 이미지 또는 비디오 스트림에서 객체를 정확하게 찾고 분류하는 것입니다.



객체 탐지

이미지 내에 존재하는 객체들을 찾아내고, 해당 객체들을 직사각형의 바운딩 박스로 둘러쌉니다.



객체 분류

탐지된 각 객체가 어떤 종류(클래스)에 속하는지 예측합니다. 예를 들어 사람, 자동차, 자전거 등을 구분합니다.



실시간 처리

다른 객체 탐지 모델에 비해 매우 빠른 추론 속도를 제공하여, 자율 주행, 비디오 감시, 로봇 공학 등 실시간 응용 분야에 적합합니다.

YOLO 버전별 진화



YOLOv10이 최고의 선택인 이유

YOLOv10은 특히 다음과 같은 상황에서 뛰어난 선택입니다:

NMS 후처리 제거

진정한 실시간 처리가 필요한 경우 NMS 후처리 오버헤드를 완전히 제거했습니다.

엣지 디바이스 최적화

추론 파이프라인을 최대한 단순화하여 엣지 디바이스 배포에 최적화되었습니다.

End-to-End 학습

완전 미분 가능한 모델로 End-to-End 학습이 가능합니다.

최고 효율성

최신 SOTA 성능과 최고 효율성을 동시에 달성했습니다.

YOLOv10의 핵심 개선 사항

개선 분야

- End-to-End
- 모델 구조
- 학습 전략
- 손실 함수

효과

- NMS-Free Detection으로 추론 지연 시간 감소
- 효율성과 정확도를 분리하여 최적화
- Dual-Path Bounding Box Matching으로 학습 안정성 향상
- Consistency-Focused Loss로 평가 지표 불일치 감소

NMS-Free Detection 혁신

전통적인 NMS의 문제점

추가 연산 시간

추론 후 별도의 후처리 단계가 필요하여 전체 지연 시간 증가

하이퍼파라미터 의존성

IoU 임계값 설정에 따라 성능이 크게 달라짐

미분 불가능

NMS는 미분 불가능한 연산이므로 End-to-End 학습이 불가능

병렬 처리 제한

GPU 병렬화가 어려워 하드웨어 가속의 이점을 충분히 활용하지 못함

YOLOv10의 One-to-One Matching 전략

기존 방식

One-to-Many: 하나의 객체 → 여러 개의 바운딩 박스 예측 → NMS로 중복 제거

YOLOv10 방식

One-to-One: 하나의 객체 → 정확히 하나의 바운딩 박스만 예측 → NMS 불필요

구현 방법

이중 헤드 구조: 학습 시에는 One-to-Many Head로 풍부한 학습 신호 제공, 추론 시에는 One-to-One Head로 각 객체당 하나의 예측만 생성

일관성 매칭 메트릭: Hungarian Algorithm과 유사한 이분 매칭을 사용하여 중복 할당 방지

중복 억제: 손실 함수에 중복 예측을 페널티로 추가하여 동일한 객체에 대한 경쟁 방지

Conda 환경 설정

Conda 환경 생성 및 활성화

Anaconda Prompt 또는 Command Prompt를 관리자 권한으로 실행한 후 다음 명령어를 순서대로 입력하세요:

```
conda create -n yolov10_env python=3.10 -y

conda activate yolov10_env
```

PyTorch 설치

CPU 버전 (권장 - 초보자용)

GPU가 없거나 CUDA 설정이 복잡한 경우 사용하세요. 성능은 다소 떨어지지만 정상

```
conda install pytorch torchvision torchaudio cpuonly -c pytorch
```

GPU 버전 (NVIDIA GPU + CUDA 11.8)

NVIDIA GPU가 있는 경우 더 빠른 성능을 위해 사용하세요.

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c
pytorch -c nvidia
```

필수 라이브러리 설치

```
pip install ultralytics

pip install opencv-python numpy matplotlib

python -c "import cv2, torch, ultralytics; print('모든 라이브러리 설치 완료')"
```

▮ 마지막 명령어에서 오류가 발생하지 않으면 모든 라이브러리가 정상 설치된 것입니다.

YOLO v10 모델 다운로드

01

weights 폴더 생성

프로젝트 루트 디렉토리에서 실행하세요:

```
mkdir weights
```

02

모델 파일 다운로드

YOLOv10n.pt 다운로드 (파일 크기: 약 5.8MB)

<https://github.com/THU-MIG/yolov10/releases/download/v1.1/yolov10n.pt>

다운로드한 yolov10n.pt 파일을 weights 폴더 안에 저장하세요.

모델 버전 선택 가이드



yolov10n.pt

가장 빠름, 정확도 보통 (권장)



yolov10m.pt

높은 정확도, 속도 보통



yolov10s.pt

균형잡힌 성능



yolov10l.pt

매우 높은 정확도, 속도 느림

단일 이미지 검출

소스코드: 101_predict_image.py

1단계: 모델 로드 및 이미지 불러오기

```
from ultralytics import YOLO
import cv2
import os# 모델 로드
model_name = './weights/yolov10n.pt'
try:
    model = YOLO(model_name)
    print(f"모델 로드 완료: {model_name}")
except FileNotFoundError:
    print(f"오류: 모델 파일을 찾을 수 없습니다.")
    exit()
# 이미지 파일 경로
image_path = './images/class.jpg'
if not os.path.exists(image_path):
    print(f"오류: 이미지 파일을 찾을 수 없습니다.")
    exit()
```

2단계: 추론 실행

```
print(f"이미지 경로: {image_path}")
img = cv2.imread(image_path)
if img is None:
    print("오류: OpenCV로 이미지를 로드할 수 없습니다.")
    exit()
print("추론 시작...")
# predict 메소드 실행
results = model.predict(
    source=img,
    conf=0.25,
    verbose=False
)
print("추론 완료.")
```

📌 `conf=0.25`는 신뢰도 임계값으로, 25% 이상의 신뢰도를 가진 객체만 탐지합니다.

3단계: 결과 시각화

```
if results:
    result = results[0]
    num_detections = len(result.bboxes)
    print(f"검출된 총 객체 수: {num_detections}")
    # 바운딩 박스, 클래스 ID, 신뢰도 추출
    bboxes = result.bboxes.xyxy.cpu().numpy()
    classes = result.bboxes.cls.cpu().numpy().astype(int)
    confidences = result.bboxes.conf.cpu().numpy()

    # 시각화 설정
    thickness = 2
    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 0.7

    # 바운딩 박스 및 레이블 그리기
```

3단계: 결과 시각화 (계속)

```
for box, cls_id, conf in zip(bboxes, classes, confidences):
    x1, y1, x2, y2 = map(int, box)
    class_name = model.names[cls_id]
    color = (255, 0, 0)
    cv2.rectangle(img, (x1, y1), (x2, y2), color, thickness)
    label = f"{class_name} ({conf:.2f})"
    (w, h), _ = cv2.getTextSize(label, font, font_scale, thickness)
    cv2.rectangle(img, (x1, y1 - h - 5), (x1 + w, y1), color, -1)
    cv2.putText(img, label, (x1, y1 - 5), font, font_scale, (255, 255, 255), thickness, cv2.LINE_AA)

# 결과 이미지 표시
cv2.imshow('YOLOv10 Custom Detection', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

# 결과 저장
output_dir = 'yolov10_output_cv'
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, os.path.basename(image_path).replace('.', '_cv_output.'))
cv2.imwrite(output_path, img)
print(f"결과 이미지 저장: {output_path}")else: print("검출된 객체가 없습니다.")

else:
    print("검출된 객체가 없음")
```

카메라 라이브 검출

실시간 웹캠 객체 탐지

1단계: 모델 로드 및 색상 함수 정의

```
from ultralytics import YOLO

import cv2

import os

model_name = './weights/yolov10n.pt'

try:
    model = YOLO(model_name)
    print(f"모델 로드 완료: {model_name}")
except FileNotFoundError:
    print(f"오류: 모델 파일을 찾을 수 없습니다.")
    exit()

def get_color_by_class(cls_id):
    """클래스 ID에 따라 고유한 BGR 색상 튜플을 반환합니다."""
    colors = [ (255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (0, 255, 255), (255, 0, 255), (128, 0, 0), (0, 128, 0), (0, 0, 128),
                (128, 128, 0), (0, 128, 128), (128, 0, 128), (255, 128, 0), (255, 0, 128), (0, 255, 128), (128, 255, 0), (128, 0, 255),
                (0, 128, 255), (255, 255, 128), (128, 255, 255) ]
    return colors[cls_id % len(colors)]
```

2단계: 웹캠 캡처 및 실시간 처리

```
# 웹캠 캡처 객체 초기화
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("오류: 카메라를 열 수 없습니다.")
    exit()
print("웹캠 실시간 객체 검출을 시작합니다. 'q' 키를 누르면 종료됩니다.")
while cap.isOpened():
    success, frame = cap.read()
    if not success:
        print("프레임을 받을 수 없습니다. 종료합니다.")
        break
    # 추론 실행
    results = model.predict(
        source=frame,
        conf=0.25,
        verbose=False,
        stream=True
    )
```

2단계: 웹캠 캡처 및 실시간 처리

```
# 결과 시각화
for result in results:
    boxes = result.bboxes.xyxy.cpu().numpy()
    classes = result.bboxes.cls.cpu().numpy().astype(int)
    confidences = result.bboxes.conf.cpu().numpy()
    thickness = 2
    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 0.7
    for box, cls_id, conf in zip(boxes, classes, confidences):
        x1, y1, x2, y2 = map(int, box)
        class_name = model.names[cls_id]
        color = get_color_by_class(cls_id)
```

2단계: 웹캠 캡처 및 실시간 처리 (계속)

```
cv2.rectangle(frame, (x1, y1), (x2, y2), color, thickness)
label = f"{class_name} ({conf:.2f})"
(w, h), _ = cv2.getTextSize(label, font, font_scale, thickness)
cv2.rectangle(frame, (x1, y1 - h - 5), (x1 + w, y1), color, -1)
cv2.putText(frame, label, (x1, y1 - 5), font, font_scale, (255, 255, 255), thickness, cv2.LINE_AA)
cv2.imshow('YOLOv10 Live Detection', frame)
if cv2.waitKey(1) & 0xFF == ord('q'): \
    break
cap.release()
cv2.destroyAllWindows()
```

이 코드 블록은 웹캠에서 실시간으로 객체를 감지하고, 감지된 객체에 바운딩 박스와 레이블을 표시한 후 화면에 출력하는 과정을 보여줍니다. 'q' 키를 누르면 프로그램이 종료됩니다.